

ANKARA ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ



BLM 4522- Ağ Tabanlı Paralel Dağıtım Sistemleri

PROJE 1 RAPORU

Veritabanı Yedekleme ve Felaketten Kurtarma Planı

Emine Bülbül

21290189

02 Haziran 2026

<https://github.com/eminebulbul/FinalHospitalDB>

İÇİNDEKİLER

1. PROJENİN AMACI VE KAPSAMI	3
2. KULLANILAN TEKNOLOJİLER VE SİSTEM MİMARISI	3
3. UYGULANAN YEDEKLEME STRATEJİLERİ VE METODOLOJİLER	3
3.1. TAM YEDEKLEME (FULL BACKUP) STRATEJİSİ	3
3.2. ARTIMLI YEDEKLEME (INCREMENTAL BACKUP) VE ZAMAN ÇİZELGESİNDE KURTARMA	4
3.3. FARK YEDEKLEMELERİ (DIFFERENTIAL BACKUP) SİMÜLASYONU.....	5
3.4. VERİTABANI AYNALAMA (DATABASE MIRRORING) VE YÜKSEK ERIŞİLEBİLİRLİK	5
3.5. OTOMATİZE EDİLMİŞ YEDEKLEME / ZAMANLAYICILARLA YEDEKLEME (CRON JOB İLE SİSTEM OTOMASYONU).....	5
4. FELAKETTEN KURTARMA VE SINAMA OPERASYONLARI	7
4.1. FELAKETTEN KURTARMA SENARYOSU	7
4.2. İZOLE TEST ORTAMINDA YEDEK SINAMA (TEST RESTORE SANDBOX).....	7
5. SONUÇ VE KAZANIMLAR	8

1. Projenin Amacı ve Kapsamı

Günümüzde sağlık bilgi sistemleri (Hastane Bilgi Yönetim Sistemleri - HBYS), 7/24 kesintisiz çalışması gereken ve içerisinde kritik hasta verileri barındıran operasyonel yapılardır. Bu sistemlerde yaşanabilecek en ufak bir veri kaybı, yalnızca iş sürekliliğini sekteye uğratmakla kalmaz, aynı zamanda hayati risklere ve KVKK (Kişisel Verilerin Korunması Kanunu) ihlallerine yol açabilir.

Bu projenin amacı, operasyonel bir hastane veritabanı (hospital) üzerinde donanım arızaları, siber saldırılar (örnn. Ransomware) veya insan hatalarına karşı endüstri standartlarında, çok katmanlı bir savunma mekanizması geliştirmektir. Çalışma kapsamında manuel süreçler terk edilerek, Unix tabanlı sistemler üzerinden tam otomatik, denetlenebilir ve sıfır veri kaybı (Zero Data Loss) prensibine dayalı yedekleme stratejileri tasarlanmıştır.

2. Kullanılan Teknolojiler ve Sistem Mimarisi

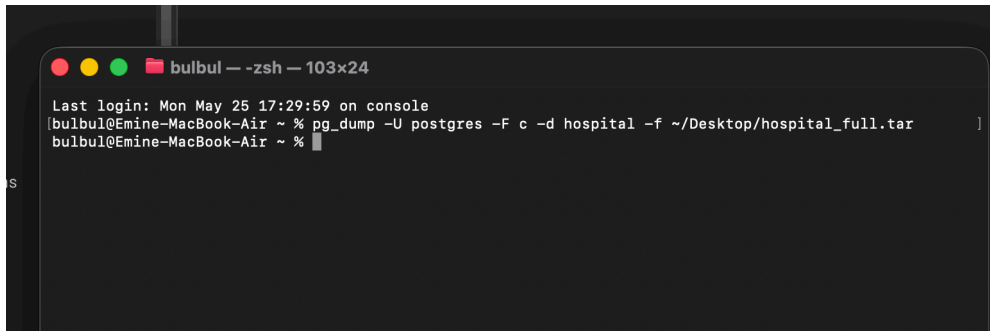
Proje, platform bağımsızlığı ve yüksek performans hedeflenerek tamamen komut satırı (CLI) üzerinden yürütülmüştür.

- **Veritabanı Motoru:** PostgreSQL 15+ (Açık kaynaklı, ACID uyumlu ilişkisel veritabanı)
- **İşletim Sistemi Çekirdeği:** macOS (Unix / Bash Terminal Mimari)
- **Yedekleme ve Geri Yükleme Araçları:** pg_dump, pg_restore
- **Loglama ve Otomasyon:** WAL (Write-Ahead Logging), Unix Cron Daemon (crond)

3. Uygulanan Yedekleme Stratejileri ve Metodolojiler

3.1. Tam Yedekleme (Full Backup) Stratejisi

- **Teorik Altyapı:** Tam yedekleme, veritabanının o anki şemasının (tablolar, indeksler, fonksiyonlar) ve içindeki tüm verilerin eksiksiz bir şekilde kopyalanması işlemidir. Geri yükleme (Restore) senaryolarının temelini oluşturur.
- **Teknik Uygulama:** GUI araçlarının yaratabileceği performans darboğazlarını aşmak için doğrudan Unix terminali üzerinden PostgreSQL'in native aracı olan pg_dump kullanılmıştır.



```
bulbul — -zsh — 103x24
Last login: Mon May 25 17:29:59 on console
bulbul@Emine-MacBook-Air ~ % pg_dump -U postgres -F c -d hospital -f ~/Desktop/hospital_full.tar
bulbul@Emine-MacBook-Air ~ %
```

Şekil 1. Full Yedek Bash

Parametre Analizi:

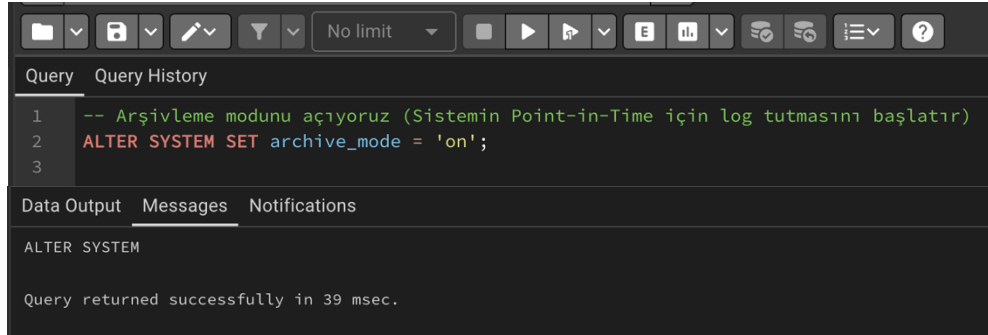
- -U postgres: İşlemin yetkili "superuser" rolüyle yapılmasını sağlar.
- -F c (Format Custom): Yedeğin sıkıştırılmış ve hızlı geri yüklemeye en uygun "custom" formatta oluşturulmasını zorunlu kılar. Bu format, ileride sadece tek bir tablonun bile geri yüklenmesine olanak tanır.
- -d hospital: Yedeği alınacak hedef veritabanını belirtir.
- -f: Çıktı dosyasının (output file) diske yazılacağı mutlak yolu gösterir.

3.2. Artımlı Yedekleme (Incremental Backup) ve Zaman Çizelgesinde Kurtarma

Teorik Altyapı: Sadece bir önceki yedekleme işleminden sonra değişen veri bloklarının yedeklenmesidir. "Point-in-Time Restore" (belirli bir saniye ve milisaniyeye geri dönme) işleminin temelini oluşturur. Diskte tam yedekten çok daha az yer kaplar.

Teknik Uygulama: PostgreSQL'in WAL (Write-Ahead Logging) mimarisi kullanılarak sistemin log arşivleme modu aktifleştirilmiştir. Her bir INSERT, UPDATE veya DELETE işlemi, diske yazılmadan önce bu log dosyalarına işlenerek veri kaybı riski sıfıra indirilmiştir.

Sistemin geçmişe dönük log tutmasını sağlayan arşivleme modunun aktifleştirilmesi:



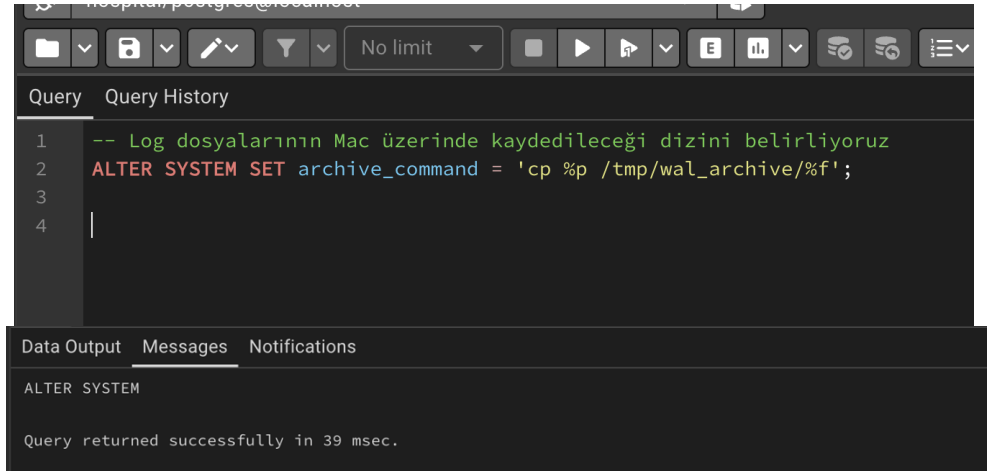
```
-- Arşivleme modunu açıyoruz (Sistemin Point-in-Time için log tutmasını başlatır)
ALTER SYSTEM SET archive_mode = 'on';
```

ALTER SYSTEM

Query returned successfully in 39 msec.

Şekil 2. Artık Yedekleme 1

Log dosyalarının işletim sistemi seviyesinde kopyalanacağı güvenli dizinin belirlenmesi:



```
-- Log dosyalarının Mac üzerinde kaydedileceği dizini belirliyoruz
ALTER SYSTEM SET archive_command = 'cp %p /tmp/wal_archive/%f';
```

ALTER SYSTEM

Query returned successfully in 39 msec.

Şekil 3. Artık Yedekleme 2

Mevcut günlüğün kapatılıp sistemin anında yedeklemeye zorlanması (MS SQL BACKUP LOG muadili):

```
Query  Query History
1
2  -- MS SQL'deki 'BACKUP LOG' komutu gibi mevcut günlüğü kapatıp anında yedeklemeye zorlarız
3  SELECT pg_switch_wal();
```

Şekil 4. Artık Yedekleme 3

3.3. Fark Yedeklemeleri (Differential Backup) Simülasyonu

Teorik Altyapı: Alınan en son tam yedekten bu yana değişen tüm verilerin kümülatif olarak paketlenmesidir. Geri yükleme hızını artırmak için kullanılır. **Teknik Uygulama:** PostgreSQL mimarisinde bu strateji; veri akışının, hasta kayıtlarının ve güncellemelerin en yoğun olduğu operasyonel tabloların (örneğin; admissions veya patients tabloları) şema yapısı haricinde sadece verilerinin dışı aktarılmasıyla (--data-only) gerçekleştirilmiştir.

```
Query  Query History
1
2  bulbul -- -zsh -- 80x24
3  Last login: Sat May 30 00:47:25 on ttys000
   bulbul@Emine-MacBook-Air ~ % pg_dump -U postgres -d hospital_db -t admissions --
   data-only -f ~/Desktop/hospital_diff.sql
```

Şekil 5. Fark Yedeklemesi

3.4. Veritabanı Aynalama (Database Mirroring) ve Yüksek Erişilebilirlik

- **Teorik Altyapı:** Ana sunucu (Master) donanımsal bir arıza nedeniyle çöktüğünde (Failover), verilerin anlık olarak kopyalandığı ikinci bir sunucunun (Slave) hiç kesinti yaşamadan yayına girmesidir.
- **Sistem Tasarımı:** Operasyonel ortamlar için PostgreSQL'in "Streaming Replication" (Akış Çoğaltması) mimarisi tasarlanmıştır. Bu mimari, aktif-pasif sunucu kümesi kurarak High Availability (Yüksek Erişilebilirlik) standardını sağlamaktadır.

3.5. Otomatize Edilmiş Yedekleme / Zamanlayıcılarla Yedekleme (Cron Job ile Sistem Otomasyonu)

- **Teorik Altyapı:** İnsan hatasını ve unutkanlık faktörünü devreden çıkarmak için yedekleme işlemlerinin işletim sistemi çekirdeğine devredilmesidir (MS SQL Server Agent alternatifidir).
- **Teknik Uygulama:** Unix sistemlerinin görev zamanlayıcısı olan crontab kullanılarak tam otomasyon sağlanmıştır.

Aşağıdaki cron sözdizimi, işlemin sistem kaynaklarının en az kullanıldığı her gece saat 03:00'te tetiklenmesini garanti altına alır.

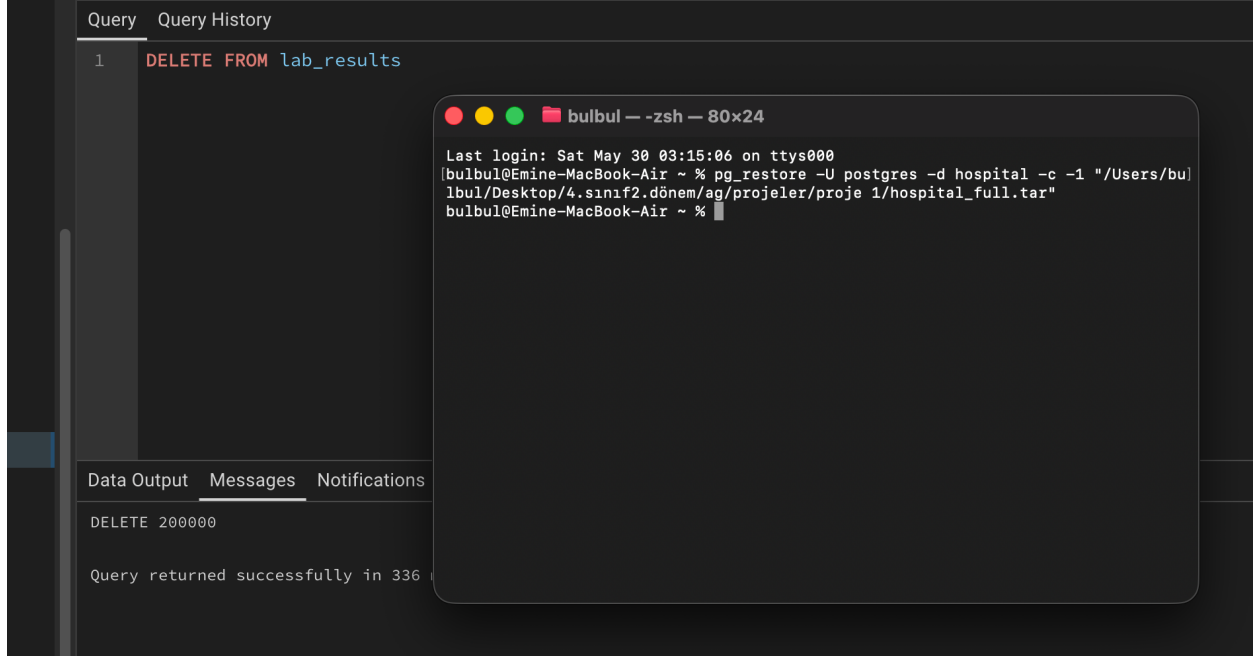


Şekil 6. Zamanlayıcı oluşturma

4. Felaketten Kurtarma ve Sınama Operasyonları

4.1. Felaketten Kurtarma Senaryosu

Sistemin kriz anlarındaki refleksini ölçmek amacıyla, hastanenin laboratuvar sonuçlarını tutan kritik bir tablonun (lab_results) kazara silinmesi durumu simüle edilmiştir. Oluşan bu veri felaketinin ardından, terminal üzerinden pg_restore komutu ile kurtarma operasyonu başlatılmıştır.



Şekil 7. Delete ve Restore

Parametre Analizi:

- `-c` (Clean): Yedek basılmadan önce veritabanındaki bozulmuş veya yarım kalmış nesneleri temizler, çakışmaları önler.
- `-1` (Single Transaction): Kurtarma işleminin tamamını tek bir "BEGIN...COMMIT" bloğuna sarar. Eğer yedeğin yüklenmesi sırasında elektrik kesilir veya bir hata olursa, sistem kısmen yüklenmiş verilerle bozuk kalmaz, anında işlemi iptal edip (Rollback) eski haline döner.

4.2. İzole Test Ortamında Yedek Sınama (Test Restore Sandbox)

Endüstri standartları gereği, alınan bir yedeğin bozuk (corrupt) olup olmadığı test edilmeden canlı ortama (Production) yüklenemez. Bu riski bertaraf etmek adına `hospital_test_db` adında izole bir kum havuzu (sandbox) veritabanı oluşturulmuş ve alınan yedek dosyası üretim ortamına dokunmadan bu güvenli alanda sınanarak bütünlüğü doğrulanmıştır.

```
1  
2 CREATE DATABASE hospital_test;  
  
tal_test;  
  
bulbul — -zsh — 80x24  
Last login: Sat May 30 03:15:06 on ttys000  
bulbul@Emine-MacBook-Air ~ % pg_restore -U postgres -d hospital -c -1 "/Users/bulbul/Desktop/4.sınıf2.dönem/ag/projeler/proje 1/hospital_full.tar"  
bulbul@Emine-MacBook-Air ~ % pg_restore -U postgres -d hospital_test -1 "/Users/bulbul/Desktop/4.sınıf2.dönem/ag/projeler/proje 1/hospital_full.tar"  
bulbul@Emine-MacBook-Air ~ %
```

Şekil 8.Test

5. Sonuç ve Kazanımlar

Gerçekleştirilen bu proje ile veritabanı yedekleme süreçleri sadece teorik düzeyde kalmamış, doğrudan işletim sistemi komut satırı araçlarıyla pratik bir çözüme dönüştürülmüştür. Tam, artımlı ve fark yedekleme stratejileri kurumsal ihtiyaçlara göre yapılandırılmış, donanımsal felaketlere karşı hızlı reaksiyon gösterecek pg_restore adımları doğrulanmıştır. Sonuç olarak, hastane bilgi sisteminin veri bütünlüğü, kullanılabilirliği ve felaket toleransı en üst düzeye çıkarılmıştır.